SIMATS ENGINEERING

ASSIGNMENT 3 – CO3

C++ Programs and Predicted Outputs (Q1–Q7)

Student Name:	_____
Register Number:	_____
Department / Section:	_____
Subject:	C++ Programming
GitHub Repository Link:	_____
Submission Date:	_____

Important submission note: This report provides source code and predicted/sample outputs based on the uploaded assignment brief. The assignment also requires you to compile and execute every task yourself, include actual output screenshots, create the specified GitHub repository, commit work progressively, and provide a README.md. Fill in your own details and GitHub link before submission.

General Compilation Instructions

Use a C++17 compiler such as g++. Save each question as a separate source file (q1.cpp, q2.cpp, etc.). Example command: **g++ -std=c++17 -Wall -Wextra q1.cpp -o q1**. Run the executable after successful compilation. For Q2 and memory-management checks, the assignment suggests AddressSanitizer or Valgrind where available.

The output blocks below are predicted outputs for the supplied sample programs. Verify them by running the code in your SIMATS lab environment.

Q1 – Constructors and Constructor Overloading

Aim / Task: Create library Book records using default, donated-book and full-detail constructors. Use member initializer lists, constructor delegation, and reject invalid copy counts.

C++ Program:

```
#include <iostream>
#include <string>
#include <stdexcept>
#include <utility>
using namespace std;

class Book {
    string title;
    string author;
    int copies;

    static int validCopies(int n) {
        if (n < 0) throw invalid_argument("Number of copies cannot be negative");
        return n;
    }

public:
    // Default constructor delegates to the full constructor.
    Book() : Book("Untitled", "Unknown", 0) {}

    // Donated book: only title is supplied.
    explicit Book(string t) : Book(move(t), "Unknown", 0) {}

    // Full-detail parameterised constructor.
    Book(string t, string a, int n)
        : title(move(t)), author(move(a)), copies(validCopies(n)) {}

    void display() const {
        cout << "Title: " << title
             << " | Author: " << author
             << " | Copies: " << copies << '\n';
    }
};

int main() {
    try {
        Book emptyBook;
        Book donatedBook("The C++ Guide");
        Book fullBook("Object Oriented Programming", "Bjarne Stroustrup", 5);

        cout << "Default record: "; emptyBook.display();
        cout << "Donated record: "; donatedBook.display();
        cout << "Full record: "; fullBook.display();

        // This test is expected to throw an exception.
        try {
            Book invalidBook("Invalid Book", "Unknown", -2);
        } catch (const invalid_argument& e) {
            cout << "Invalid record rejected: " << e.what() << '\n';
        }
    } catch (const exception& e) {
        cout << "Error: " << e.what() << '\n';
    }
    return 0;
}

/*
Explicit-conversion demonstration:
void registerBook(Book b) {}
registerBook("C++ Basics"); // Does not compile because Book(string) is explicit.
registerBook(Book("C++ Basics")); // Valid explicit construction.
*/
```

Predicted Output:

```
Default record: Title: Untitled | Author: Unknown | Copies: 0
Donated record: Title: The C++ Guide | Author: Unknown | Copies: 0
Full record: Title: Object Oriented Programming | Author: Bjarne Stroustrup | Copies: 5
Invalid record rejected: Number of copies cannot be negative
```

Design and Analysis Notes:

• Constructor delegation lets the default and donated-book constructors reuse the full constructor, keeping validation in one place.

• A compiler-generated default constructor is supplied only when no constructor has been declared by the programmer. Once a constructor is declared, the compiler does not automatically add a no-argument constructor.

• Member initializer lists initialise members directly and are required for const members and references. Members are initialised in the order in which they are declared in the class, not the order written in the initializer list.

• The single-argument constructor is explicit to prevent unintended implicit conversion from string to Book. Object creation and validation take O(1) time and O(1) extra space.

Q2 – Copy Constructors: Shallow and Deep Copies

Aim / Task: Complete the Prescription class with a deep-copy constructor, copy assignment operator and destructor so that copied prescriptions own independent medicine arrays.

C++ Program:

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(new string[count]) {}

    // TODO 1: Deep-copy constructor
    Prescription(const Prescription& other)
        : n(other.n), medicines(new string[other.n]) {
        for (int i = 0; i < n; ++i)
            medicines[i] = other.medicines[i];
    }

    // TODO 2: Copy assignment operator
    Prescription& operator=(const Prescription& other) {
        if (this != &other) {
            string* newMedicines = new string[other.n];
            for (int i = 0; i < other.n; ++i)
                newMedicines[i] = other.medicines[i];

            delete[] medicines;
            medicines = newMedicines;
            n = other.n;
        }
        return *this;
    }

    // TODO 3: Destructor
    ~Prescription() {
        delete[] medicines;
    }

    void set(int i, const string& m) { medicines[i] = m; }
    string get(int i) const { return medicines[i]; }
};

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p;       // Copy constructor
    q.set(1, "Cetirizine");

    Prescription r(1);
    r = p;                     // Copy assignment

    p.set(0, "Ibuprofen");

    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\n';
    return 0;
}
```

Predicted Output:

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

Design and Analysis Notes:

• A shallow copy copies only the pointer value. Both objects then refer to the same array; modifying one changes the other, and both destructors may try to delete the same memory (double delete).

• A deep copy allocates a new array and copies each medicine name, so each object owns its own memory.

• Rule of Three: if a class needs a user-defined destructor, it commonly also needs a copy constructor and copy assignment operator. The copy constructor is used when an object is initialised from another object, passed by value, or returned by value (subject to copy elision).

• Copying n medicine names takes O(n) time and O(n) additional space. A std::vector member follows the Rule of Zero and safely provides value-copy behaviour without manually writing these three functions.

• To compile with AddressSanitizer where supported: g++ -std=c++17 -Wall -Wextra -fsanitize=address -g q2.cpp -o q2

Q3 – Dynamic Constructors and Destructors

Aim / Task: Predict the object construction and destruction order for local, block-scope, copied and dynamically allocated Logger objects.

C++ Program:

```
#include <iostream>
#include <string>
using namespace std;

struct Logger {
    string n;

    Logger(const string& s) : n(s) {
        cout << '+' << n << ' ';
    }

    Logger(const Logger& other) : n(other.n + "c") {
        cout << "copy" << n << ' ';
    }

    ~Logger() {
        cout << '-' << n << ' ';
    }
};

void use(Logger) {
    cout << "use ";
}

int main() {
    Logger a("A");

    {
        Logger b("B");
    }

    use(a);

    Logger* p = new Logger("P");
    delete p;

    cout << "end ";
    return 0;
}
```

Predicted Output:

```
+A +B -B copyAc use -Ac +P -P end -A
```

Design and Analysis Notes:

• Logger a is constructed first and remains alive until main ends. Logger b is constructed inside the inner block and destroyed when that block ends.

• use(a) takes its parameter by value, so a copy named Ac is constructed. The parameter is destroyed when use returns.

• The dynamically allocated Logger P is constructed by new and destroyed when delete p is executed.

• At the end of main, the local object a is destroyed. Local objects are destroyed in reverse order of completed construction within their scope.

• The destructor has no parameters and cannot be overloaded. Forgetting delete for a dynamically allocated object causes a memory leak. Arrays allocated with new[] must be released with delete[].

Q4 – Overloading Arithmetic and Stream Operators

Aim / Task: Complete the Money class by overloading +, == and <<. Store the value as paise to avoid binary floating-point representation issues in basic money arithmetic.

C++ Program:

```
#include <iomanip>
#include <iostream>
using namespace std;

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}

    // TODO 1: Addition returns a new Money object.
    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }

    // TODO 2: Equality operator.
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }

    // Stream insertion is a non-member friend.
    friend ostream& operator<<(ostream& out, const Money& m) {
        out << "Rs " << m.paise / 100 << '.'
            << setfill('0') << setw(2) << m.paise % 100
            << setfill(' ');
        return out;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;

    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';

    Money x(0, 75), y(0, 50);
    cout << "Carry test: " << x + y << '\n';
    return 0;
}
```

Predicted Output:

```
Rs 200.00
true
Carry test: Rs 1.25
```

Design and Analysis Notes:

• operator+ returns a new Money object, while operator+= would normally modify the left operand and return Money& (*this) to allow chaining.

• operator<< is a non-member because the left operand is an ostream object. It is declared friend so it can access the private paise member.

• The expression a + b is interpreted as a.operator+(b) for this member-operator design. These operations take O(1) time and O(1) extra space.

• The class stores paise as an integer, avoiding the common precision problems of storing decimal currency directly in double. This example assumes non-negative amounts for display.

Q5 – Overloading Increment, Comparison and Subscript Operators

Aim / Task: Predict the values produced by prefix and postfix increment in the given Counter program.

C++ Program:

```
#include <iostream>
using namespace std;

struct Counter {
    int v;

    Counter& operator++() {       // Prefix
        ++v;
        return *this;
    }

    Counter operator++(int) {      // Postfix
        Counter old = *this;
        ++v;
        return old;
    }
};

int main() {
    Counter c{5};

    Counter a = c++;
    Counter b = ++c;

    cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
    return 0;
}
```

Predicted Output:

```
5 7 7
```

Design and Analysis Notes:

• Initially c.v is 5. The postfix expression c++ returns the old value 5 into a, then increments c to 6.

• The prefix expression ++c increments c from 6 to 7 first and returns the updated object, so b.v is 7. The final c.v is also 7.

• The dummy int parameter distinguishes postfix operator++(int) from prefix operator++(). Postfix generally creates and returns a copy of the old value; prefix returns a reference to the updated object.

• Prefix and postfix increment are O(1) for this small class. The assignment also asks for Time comparison and checked const/non-const Playlist operator[] overloads; those should be added and tested if your instructor expects the complete implementation section.

Q6 – The Rules of Operator Overloading

Aim / Task: Implement a Text class with dynamic character storage, copy construction, copy assignment, destruction, concatenation and comparison operators.

C++ Program:

```
#include <iostream>
#include <cstring>
using namespace std;

class Text {
    char* data;

public:
    Text(const char* s = "") {
        data = new char[strlen(s) + 1];
        strcpy(data, s);
    }

    // Deep-copy constructor
    Text(const Text& other) {
        data = new char[strlen(other.data) + 1];
        strcpy(data, other.data);
    }

    // Copy assignment with self-assignment protection
    Text& operator=(const Text& other) {
        if (this != &other) {
            char* temp = new char[strlen(other.data) + 1];
            strcpy(temp, other.data);
            delete[] data;
            data = temp;
        }
        return *this;
    }

    Text operator+(const Text& other) const {
        char* temp = new char[strlen(data) + strlen(other.data) + 1];
        strcpy(temp, data);
        strcat(temp, other.data);

        Text result(temp);
        delete[] temp;
        return result;
    }

    bool operator==(const Text& other) const {
        return strcmp(data, other.data) == 0;
    }

    bool operator!=(const Text& other) const {
        return !(*this == other);
    }

    friend ostream& operator<<(ostream& out, const Text& t) {
        return out << t.data;
    }

    ~Text() {
        delete[] data;
    }
};

int main() {
    Text first("Hello ");
    Text second("World");
    Text combined = first + second;
    Text copy = combined;

    cout << "Combined: " << combined << '\n';
    cout << "Copy: " << copy << '\n';
    cout << "combined == copy: " << boolalpha
         << (combined == copy) << '\n';
    cout << "first != second: " << (first != second) << '\n';

    combined = combined; // Self-assignment test
    cout << "After self-assignment: " << combined << '\n';
    return 0;
}
```

Predicted Output:

```
Combined: Hello World
Copy: Hello World
combined == copy: true
first != second: true
After self-assignment: Hello World
```

Design and Analysis Notes:

• Operators that cannot be overloaded include ::, ., .*, ?: and sizeof. Operator overloading cannot change precedence, associativity or the number of operands, and at least one operand must be a user-defined type.

• operator= must be a member function. operator<< is commonly a non-member friend because its left operand is the output stream.

• The self-assignment check prevents deleting the object's own data before copying it. The Rule of Three is implemented through the destructor, copy constructor and copy assignment operator.

• Concatenation allocates enough space for both strings and copies their characters. For strings of lengths n and m, this implementation takes O(n+m) time and O(n+m) temporary space.

Q7 – Design a Complex-Number Class from First Principles

Aim / Task: Build a Complex class for AC impedance calculations. Support default, real-only and real-plus-imaginary construction; overload arithmetic, equality and output; and safely detect division by zero.

C++ Program:

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

class Complex {
    double real, imag;

public:
    Complex() : real(0), imag(0) {}
    Complex(double r) : real(r), imag(0) {}
    Complex(double r, double i) : real(r), imag(i) {}

    Complex operator+(const Complex& c) const {
        return Complex(real + c.real, imag + c.imag);
    }

    Complex operator-(const Complex& c) const {
        return Complex(real - c.real, imag - c.imag);
    }

    Complex operator*(const Complex& c) const {
        return Complex(real*c.real - imag*c.imag,
                       real*c.imag + imag*c.real);
    }

    Complex operator/(const Complex& c) const {
        double denominator = c.real*c.real + c.imag*c.imag;
        if (denominator == 0.0)
            throw runtime_error("Division by zero impedance");

        return Complex((real*c.real + imag*c.imag) / denominator,
                       (imag*c.real - real*c.imag) / denominator);
    }

    bool operator==(const Complex& c) const {
        return real == c.real && imag == c.imag;
    }

    double magnitude() const {
        return sqrt(real*real + imag*imag);
    }

    friend ostream& operator<<(ostream& out, const Complex& c) {
        out << c.real;
        if (c.imag >= 0) out << " + " << c.imag << "i";
        else out << " - " << -c.imag << "i";
        return out;
    }
};

int main() {
    try {
        Complex z1(4, 3), z2(2, -1);

        Complex series = z1 + z2;
        Complex parallel = (z1 * z2) / (z1 + z2);

        cout << "Z1 = " << z1 << '\n';
        cout << "Z2 = " << z2 << '\n';
        cout << "Series impedance = " << series << '\n';
        cout << "Parallel impedance = " << parallel << '\n';
        cout << "Magnitude of Z1 = " << z1.magnitude() << '\n';
        cout << "Z1 == Z2: " << boolalpha << (z1 == z2) << '\n';

        // Division-by-zero verification
        try {
            Complex invalid = z1 / Complex(0, 0);
        } catch (const runtime_error& e) {
            cout << "Zero test: " << e.what() << '\n';
        }
    } catch (const exception& e) {
        cout << "Error: " << e.what() << '\n';
    }
    return 0;
}
```

Predicted Output:

```
Z1 = 4 + 3i
Z2 = 2 - 1i
Series impedance = 6 + 2i
Parallel impedance = 1.9 + 0.7i
Magnitude of Z1 = 5
Z1 == Z2: false
Zero test: Division by zero impedance
```

Design and Analysis Notes:

• For z = a + bi and w = c + di: addition is (a+c)+(b+d)i; subtraction is (a-c)+(b-d)i; multiplication is (ac-bd)+(ad+bc)i.

• Division uses the denominator c^2+d^2 and checks it before calculating. The magnitude is sqrt(a^2+b^2). Each arithmetic operation takes O(1) time and O(1) space.

• The parallel impedance formula used is (Z1 × Z2)/(Z1 + Z2). std::complex provides standard complex arithmetic and is preferable in production code; a custom class is useful here to demonstrate operator design and explicit error handling.

Final Submission Checklist

- ■ Fill in student name, register number, submission date and working GitHub repository link.
- ■ Create the repository using the exact naming format required in the assignment: DSA01-<RegisterNumber>-Assignment3.
- ■ Create one folder for each task (Task01–Task07 or Q1–Q7) and include the C++ source files.
- ■ Compile and execute every program individually; capture genuine output screenshots.
- ■ For Q2, run the deep-copy implementation with AddressSanitizer or Valgrind if available.
- ■ Add a README.md with compiler/version, exact compile commands, run steps and a short result for each task.
- ■ Commit work progressively, as the assignment brief requires at least one commit for each task.
- ■ Add the GitHub link to the report cover page and submit the report, code and required evidence through Google Classroom.

Reminder: Predicted output is not a substitute for actual execution evidence. The assignment brief states that every student must execute every task individually and may be asked to explain the work in a verification discussion or viva.

— End of Report —